

# Exploring Statistical Trends in Archery Simulations

Anthony Kupshik, Clara Thompson, Eve Williamson  
and Megan Sanderson

MA2760: Mathematical Investigations with Python  
School of Mathematics,  
Cardiff University.

March 21, 2025

## 1 Introduction

This report aims to explore the statistical trends generated by a Python-built two-player archery simulation, with a focus on investigating how factors such as shot variability influence performance, accuracy and precision.

The simulation models an archery match where two players compete by shooting arrows with random variations in precision, determined by normal distributions. The statistical data generated enables the analysis of trends, such as average distances to the bullseye and their relationship with performance metrics.

The report begins by describing the underlying mechanics of the simulation and its statistical methodology. It then investigates key findings, such as the relationship between shot variability and accuracy. The report concludes with an evaluation of the simulation's implications and potential extensions for future analysis.

## 2 Functionality

This section describes the core functionality of the archery simulation. The two key functions, `playRound` and `playMatch`, form the backbone of the simulation. Each serves a specific role: `playRound` simulates individual rounds of the match, while `playMatch` oversees the full match, ensuring compliance with the winning conditions.

### 2.1 `playRound`

The `playRound` function models a single round of the match by simulating three arrows fired by each player.

The positions of these arrows, modelled as `x` and `y` coordinates, are randomly generated with mean 0 and standard deviations inputted by the user. These are subsequently added to lists `arrows_p1`, and `arrows_p2` respectively.

The euclidean distance of each arrow to the origin (0,0) is then calculated using Pythagorean Theorem, and stored in the variable `distance`. The arrow coordinates, (`x` and `y`), the distance, and player number are all appended to the list `all_arrows` in the form of a tuple.

An example tuple for an arrow fired by Player 1: (`x`, `y`, `distance`, 1)

To determine the arrow closest to the origin, The `min` function goes through each tuple in `all_arrows` and applies the lambda function to extract the distance from each arrow. The distance with the smallest value is selected, and its respective tuple is assigned to the variable `winning_arrow`. This ensures that details of the winning arrow, including the player, can be easily extracted by indexing to retrieve any element in the tuple.

## 2.2 playMatch

The `playMatch` function models an entire match by repeatedly simulating rounds until one player reaches at least 11 points and leads by at least 2 points. In each round, the `playRound` function is called to simulate the players firing three arrows each, with closest arrow to the centre determines the round's winner. The function updates the score based on the round outcome, stores detailed round information, and continues this process until the match-winning conditions are satisfied. Finally, it compiles and returns a summary of the match, including the match number (to be assigned later), the overall winner, the score, and the data for each round played.

## 2.3 simulateMatches

The `simulateMatches` function executes a Monte Carlo style simulation of multiple archery matches. It repeatedly calls the `playMatch` function for a specified number of matches, assigning a match number to each match and compiling the results into a list. Each match result includes detailed data such as the match winner, round-by-round performance, and the final score.

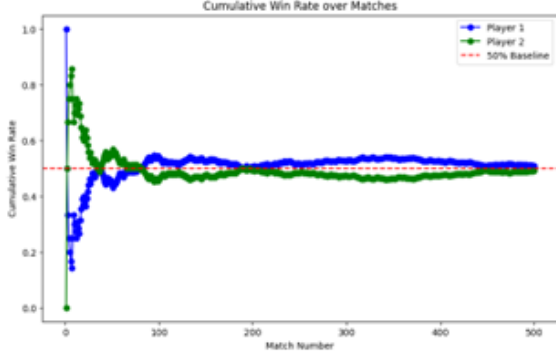
We use the aggregated simulation data for further analysis such as calculating win rates and visualizing performance trends over time, by sorting matches chronologically and computing cumulative statistics.

# 3 Investigating Trends

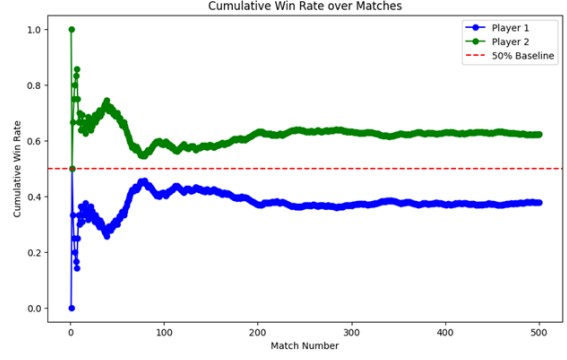
## 3.1 The effect of standard deviation on performance metrics

### 3.1.1 Cumulative Win Rate

The function `simulateMatches` was used twice to run 500 matches. One set of matches was a control where players have equal standard deviations. The other set was where Player 1 had 1.5 standard deviation, and Player 2 had 1.4 Standard deviation.



(a) Player 1: 1.5, 1.5. Player 2: 1.5, 1.5



(b) Player 1: 1.5, 1.5. Player 2: 1.4, 1.4.

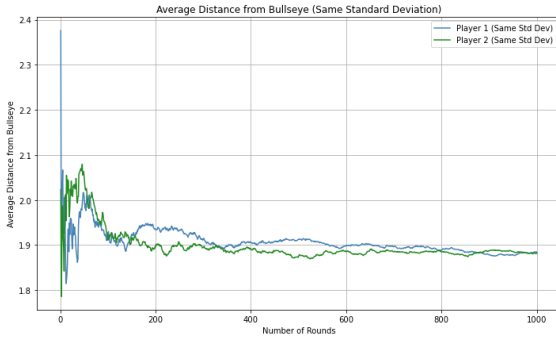
Figure 1: Comparison of cumulative win rate with varying standard deviations.

When Standard Deviations are equal, we see that both players cumulative win rate hovers around 50%. This is what you would expect when the players have identical performance levels.

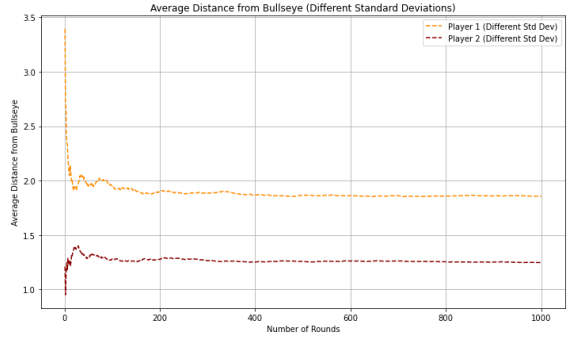
However when the players have different standard deviations, we see that the player with lower standard deviation (more consistent shooting) tends to dominate over time.

### 3.1.2 Average Distance From Bullseye

The function `simulateMatches` was used twice to run 1000 matches. One set of matches was a control where players have equal standard deviations. The other set was where Player 1 had 1.5 standard deviation, and Player 2 had 1.0 Standard deviation.



(a) Player 1: 1.5, 1.5. Player 2: 1.5, 1.5



(b) Player 1: 1.5, 1.5. Player 2: 1.0, 1.0.

Figure 2: Comparison of average distance from bullseye with varying standard deviations.

Figure 2 shows that when the two players have the same standard deviation, their average distances from the bullseye eventually converge to roughly the same level. Given enough rounds, the small differences even out and they end up performing about equally.

However, when two players have different standard deviation, the player with the lower standard deviation (the more “consistent” shots) continues to score substantially better over the entire course of matches. Even though the “less consistent” player might briefly improve and decrease the average distance, the overall gap never closes once the system settles, favouring the more consistent shooter in the long term.

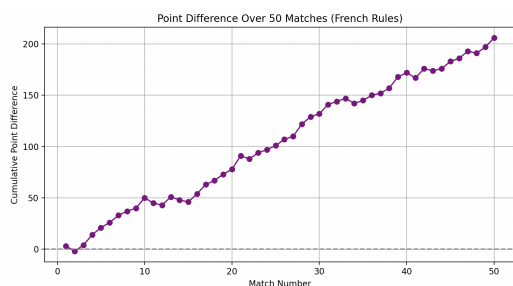
### 3.2 The effect of different rules on performance metrics

In order to gauge how different rules effect performance statistics, we simulated matches with French rules and South Korean rules.

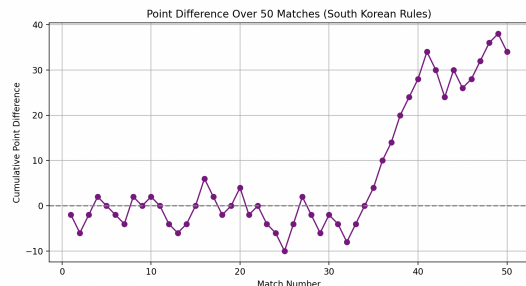
Under the French rules of archery, players accumulate points continuously over the course of a match. Each arrow’s score contributes directly to a running total, so a player with even a slight edge in skill can steadily build a considerable lead.

By contrast, the South Korean rules employ a set-based system: each set is awarded to the player with the higher total in that set, and the overall match hinges on who wins the majority of sets. This approach effectively “resets” the score every few arrows, so it often keeps the competition close until one archer conclusively demonstrates superior performance over multiple sets. If both players remain tied on set points by the final set, a one-arrow shoot-off is used as a tiebreaker. This shoot-off intensifies the competition because even a very close match can be decided on a single arrow, awarding victory to whichever arrow lands closest to the centre.

To investigate how these rules impact match outcomes, a fixed number of matches were simulated under each scoring system. For each match, the shot outcomes were recorded, and calculated the round or set winners based on the respective rules, and accumulated the point differences across the entire series of games. This allowed for a direct comparison of how quickly and how strongly one player might lead under the two different scoring methods.



(a) Player 1: 1.5, 1.6. Player 2: 1.7, 1.8



(b) Player 1: 1.5, 1.6. Player 2: 1.7, 1.8

Figure 3: Comparison of point difference over 50 matches under French rules and South Korean rules.

Figure 3 shows that with French rules, the cumulative point difference grows steadily and substantially across the 50 matches. Initially, the point gap hovers around zero, but after only a few matches it begins to climb at a fairly consistent rate. By the final matches, the leading player has a difference exceeding 200 points. This indicates that under the French scoring system, once a small advantage emerges for one player, it tends to build up continuously from match to match.

In contrast, the South Korean rules show a much more volatile and gradual cumulative point difference through the first 30 matches, with the gap often hovering near zero. This suggests a more balanced back-and-forth competition early on. However, a clear turning point occurs around match 30, after which one player steadily accumulates points and pulls ahead by roughly 35–40 points by match 50. Although a lead does eventually form, it is much smaller in size compared to the French Rules. This pattern reflects how the South Korean set-based scoring can keep the match balanced for longer and reduce the likelihood of runaway leads unless a single player outperforms the other repeatedly.

## 4 Enhancing the Realism of the Simulation

As a final part of this work, we extended our code to adapt the static model and simulate more dynamic gameplay. The `main` function acts as the entry point of the program for the terminal user and prompts them to choose between a simple or dynamic simulation. Depending on the user choice, it calls either the `playMatch` or the `playMatchWithOptions` function. In the simple simulation, static parameters such as accuracy and environmental conditions remain unchanged, allowing for a straightforward, rule-based match between players (with the choice of standard/French/South Korean rules implemented). Conversely, the dynamic simulation introduces the following factors for the user:

- Player nationalities.
- Match location.
- Fatigue, recovery and confidence factors.

At the start of a match, the weather conditions of the selected location are copied into `location_conditions`. These environmental factors directly influence player performance by modifying their shooting accuracy, a process handled by the `adjustModeForEnvironment` function. To create a dynamic experience, the `updateWeather(location_conditions.copy())` function is called at the start of each round, introducing slight, realistic changes to the weather over time.

Each player's nationality plays a pivotal role in determining how well they adapt to different environmental conditions. Nationalities are associated with specific affinities that influence their performance in various scenarios:

- Rain: Measures how effectively a player performs in rainy conditions.
- Cold: Determines their resilience and precision in cold temperatures.
- Heat: Reflects their ability to handle hot environments.

These affinities are incorporated into `adjustModeForEnvironment()` to fine-tune the accuracy mode:

- When temperatures are cold ( $<13^{\circ}\text{C}$ ), the player's cold affinity reduces the negative impact on their accuracy.
- When temperatures are hot ( $>15^{\circ}\text{C}$ ), the heat affinity enhances their performance.
- If it is raining, the rain affinity mitigates accuracy penalties caused by adverse conditions.

For example, a player from Mongolia, accustomed to cold but unaccustomed to heat, may struggle in hotter climates but excel in colder ones.

In addition to environmental factors, fatigue plays a significant role in influencing gameplay. Players begin with a fatigue level of 0, which progressively increases each round due to multiple factors: a. A base fatigue increment of 1.0 added every round. b. Higher fatigue levels for players whose shots land further from the centre. c. Random events during the match that impose additional strain. d. A user-defined fatigue sensitivity factor that determines how strongly each player is impacted by fatigue. Players recover a portion of their fatigue at the end of each round, as determined by user settings.

Finally, confidence is another key variable that affects performance. Winning rounds increases a player's confidence, improving their accuracy, while losing reduces it. These confidence shifts are incorporated into the `adjustModeForEnvironment()` function, dynamically influencing a player's shooting precision as the match progresses.

In addition, the function `randomEvent()` adds an element of unpredictability to the game by returning a randomly selected event from a list of five possibilities, such as a sudden wind gust or crowd noise. These events can temporarily impact player fatigue, confidence, or accuracy during the match.

The `playRoundTriangular` function introduces a significant improvement over the `playRound` function by incorporating a triangular distribution to generate arrow positions. Unlike the Gaussian distribution used in `playRound`, which assumes symmetric randomness around a fixed mean, the triangular distribution allows for a more flexible and adaptive model. By dynamically adjusting the mode (the most likely value for shot accuracy) `playRoundTriangular` can reflect the real-world factors added, allowing the function to simulate varied player performance and environmental interactions in real time. Additionally, the triangular distribution provides tighter control over accuracy, favoring shots closer to the center while still allowing for occasional deviations, which aligns better with the nuances of human behavior under changing conditions.

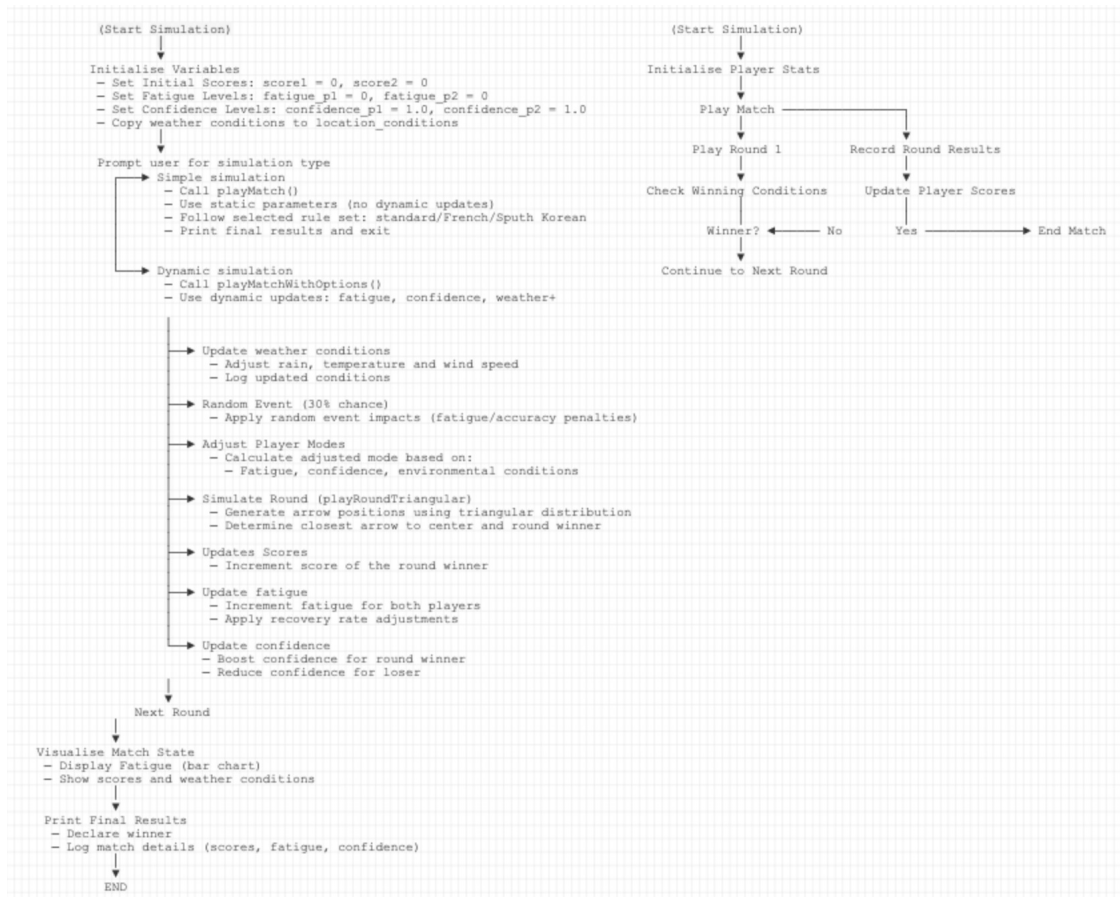


Figure 4: Comparison of dynamic (left) and simple programming (right)

## 5 Conclusion

This report has documented our exploration of statistical trends within a two-player archery simulation developed in Python. Using functions such as `playRound`, `playMatch` and `simulateMatches`, the simulation demonstrates how shot variability significantly influences player performance. Players with lower standard deviations consistently demonstrate better accuracy and consistency, leading to better overall performance.

The investigation also reveals how different scoring systems can affect match outcomes. French rules result in progressively widening score differentials for more skilled players, while South Korean rules maintain balanced competition with their set-based structure until skill differences are consistently proven. The adaptable framework provides a foundation for further enhancements, including environmental factors and player-specific traits. A real-world application for this could be implementing a dynamic player adaptation system that models responses to competitive pressure. This could lead to a practical training and performance prediction tool calibrated with professional competition data.

## Code Listing

```

1
2 import random
3 import math
4 import matplotlib.pyplot as plt
5
6 def playRound(p1x, p1y, p2x, p2y):
7     """
8     Simulates a single round of archery for two players.
9
10    Each player shoots three arrows. Each arrow's x and y coordinates are
11    generated using a normal (Gaussian) distribution centered at 0 with the
12    provided standard deviations (p1x, p1y for player 1 and p2x, p2y for player 2).
13
14    Returns:
15        dict: Contains the round number (to be added later), the winning player's number,
16              the coordinates of the winning arrow, and the distance of that arrow from the
17              center.
18    """
19    # Generate three arrow positions for each player using gaussian distributions
20    arrows_p1 = [(random.gauss(0, p1x), random.gauss(0, p1y)) for _ in range(3)]
21    arrows_p2 = [(random.gauss(0, p2x), random.gauss(0, p2y)) for _ in range(3)]
22
23    all_arrows = []
24
25    # Calculate distance for player 1's arrows and store with "1" to signal Player 1
26    for x, y in arrows_p1:
27        distance = math.hypot(x, y)
28        all_arrows.append((x, y, distance, 1))
29
30    # Calculate distance for player 2's arrows and store with "2" to signal Player 2
31    for x, y in arrows_p2:
32        distance = math.hypot(x, y)
33        all_arrows.append((x, y, distance, 2))
34
35    # Determine the arrow that is closest to the centre
36    winning_arrow = min(all_arrows, key=lambda arrow: arrow[2])
37
38    return {
39        "round_number": None, # To be set by playMatch
40        "round_winner": winning_arrow[3],
41        "winning_arrow": (winning_arrow[0], winning_arrow[1]),
42        "winning_arrow_distance": winning_arrow[2]
43    }
44
45 def playMatch(p1x, p1y, p2x, p2y):
46     """
47     Simulates an archery match between two players.
48
49     Rounds are played until one player scores at least 11 points and leads by at least 2
50     points.

```

```

50 Returns:
51     dict: Contains the match number (to be set later), the match winner, details for
52         each round,
53         and the final score.
54     """
55     score1, score2 = 0, 0
56     round_num = 1
57     rounds_data = []
58
59     # Continue playing rounds until the winning condition is met
60     while not ((score1 >= 11 or score2 >= 11) and abs(score1 - score2) >= 2):
61         round_result = playRound(p1x, p1y, p2x, p2y)
62         round_result["round_number"] = round_num # Record the round number
63
64         # Update the score based on the winner of the round
65         if round_result["round_winner"] == 1:
66             score1 += 1
67         else:
68             score2 += 1
69
70         rounds_data.append(round_result)
71         round_num += 1
72
73     match_winner = 1 if score1 > score2 else 2
74
75     return {
76         "match_number": None, # To be set by simulateMatches
77         "winner": match_winner,
78         "rounds": rounds_data,
79         "score": (score1, score2)
80     }
81
82 def simulateMatches(num_matches, p1x, p1y, p2x, p2y):
83     """
84     Simulates a series of archery matches between two players.
85
86     Args:
87         num_matches (int): The number of matches to simulate.
88         p1x (float): Standard deviation for player 1's arrow x-coordinate.
89         p1y (float): Standard deviation for player 1's arrow y-coordinate.
90         p2x (float): Standard deviation for player 2's arrow x-coordinate.
91         p2y (float): Standard deviation for player 2's arrow y-coordinate.
92
93     Returns:
94         list: A list of dictionaries, each containing data for one match.
95     """
96     matches_data = []
97     for match_num in range(1, num_matches + 1):
98         match_result = playMatch(p1x, p1y, p2x, p2y)
99         match_result["match_number"] = match_num # Set the match number
100         matches_data.append(match_result)
101
102     return matches_data

```